File System Operations - Questions with Answers

---

1. What is the difference between synchronous and asynchronous file operations?

Synchronous file operations execute one after another and block the program execution until the task is completed. The program waits for the file operation to finish before moving to the next line of code. Asynchronous file operations do not block execution. They allow the program to continue running while the file task completes in the background, usually using callbacks, promises, or async/await. Asynchronous operations are preferred in applications where performance and responsiveness are important.

2. When should you use file streams instead of reading the entire file?

File streams should be used when working with large files or continuous data. Streams process data in small chunks instead of loading the entire file into memory. This improves performance and reduces memory usage. They are ideal for large logs, videos, or when transferring data over a network.

3. Explain the purpose of the 'utf8' encoding parameter in file operations.

The 'utf8' encoding parameter specifies how text data is interpreted or written. UTF-8 is a character encoding standard that supports most languages and symbols. When reading a file, specifying 'utf8' ensures the file content is returned as a readable string instead of raw binary data.

4. What are the common error codes in file system operations and what do they mean?

Common error codes include: ENOENT (Error NO ENTry) – File or directory does not exist. EACCES – Permission denied. EEXIST – File already exists. ENOTDIR – A component of the path is not a directory. These codes help identify the specific reason for failure in file operations.

5. How would you safely delete a directory with all its contents?

To safely delete a directory with all its contents, first ensure that the correct path is provided to avoid accidental data loss. Use a recursive delete method that removes all files and subdirectories inside the directory. It is also recommended to handle errors and confirm deletion before executing the operation.

6. Explain the concept of piping in streams with an example.

Piping in streams refers to connecting the output of one stream directly to the input of another. It allows efficient data transfer between streams without storing the entire data in memory. For example, reading data from a file stream and piping it directly to a writable stream to copy a file.

7. Why is it important to handle errors in file operations?

Error handling prevents application crashes and ensures reliability. File operations may fail due to missing files, permission issues, or hardware problems. Proper error handling allows the program to respond gracefully, log issues, and inform the user appropriately.

8. What is the difference between writeFile and appendFile methods?

The writeFile method creates a new file or overwrites the existing file content. If the file already exists, its data is replaced. The appendFile method adds new data to the end of the file without deleting the existing content.